Documentación técnica del código de Team bici

Sistema web y Android para la gestión de alquiler de bicicletas

Facultad de Ingeniería, Diseño e Innovación

Autores:

Daniel Santiago García Figueroa

Tomas Soto Ortiz

Anderson Giovanny Lozano Medina

Docente:

Edilberto Torres Ortiz

Practica aplicada a sistemas

Bogotá, Colombia

1 de octubre de 2026

INDICE

1. RESUMEN 3

2. DESCRIPCIÓN GENERAL 3

2.1. Propósito del sistema 3

2.2. Problema que resuelve 4

2.3. Problema que resuelve 4

2.4. Tecnologías utilizadas 5

3. ARQUITECTURA Y ESTRUCTURA DEL PROYECTO 5

3.1. Arquitectura backend por capas 5

3.2. Estructura fronted 7

3.3. Arquitectura backend por capas 7

4. COMPONENTES Y CLASES PRINCIPALES 8

4.1. Controladores REST 8

4.2. Componentes de seguridad 8

4.3. Servicios de negocio 9

4.4. Controladores REST 10

5. MODELO DE DATOS Y PERSISTENCIA 10

5.1. Entidades y campos principales 11

5.2. Estados controlados 12

6. API REST y métodos importantes 12

6.1. Endpoints de autentificación 13

6.2. Endpoints de catálogo y administración 13

6.3. Endpoints de reservas y alquileres 14
6.4. Ejemplos de métodos críticos 14
7. HISTORIAS DE USUARIO 15
7.1. Autenticación local y BCrypt 15
7.2. JWT y control de sesión 16
7.3. Refresh tokens rotativos 16
7.4. Autorización y CORS 16
7.5. Errores seguros 17
8. APLICACIÓN CLIENTE Y EXPERIENCIA DE USUARIO 17
8.1. Comunicación y sesión 17
8.2. Experiencia de reserva 17
8.3. Diseño y retroalimentación 18
8.4. Accesibilidad y mejoras previstas 18
9. APLICACIÓN CLIENTE Y EXPERIENCIA DE USUARIO 19
9.1. Requisitos 19
9.2. Variables de entorno del backend 19
9.3. Ejecución del backend 20
9.4. Ejecución del frontend 20
10. CONVENCIONES DE CÓDIGO Y COMENTARIOS 21
10.1. Convenciones aplicadas 21
10.2. Política de comentarios 22
10.3. Ejemplo de comentario útil 22
11. VALIDACIÓN, ERRORES Y SONSIDERACIONES OPERATIVAS 22

11.1. Validación de datos 22

11.2. Reglas de integridad destacadas 23

11.3. Operación y evolución recomendada 23

12. REFERENCIAS 24

1. RESUMEN

Este documento describe la arquitectura, los componentes de software, el modelo de datos, la interfaz de programación de aplicaciones (API), los controles de seguridad y el procedimiento de ejecución de Team Bici. El sistema permite a clientes registrarse, autenticarse, consultar bicicletas disponibles, reservar bicicletas por intervalos de tiempo, iniciar alquileres y efectuar devoluciones. A su vez, las personas con rol administrador pueden gestionar estaciones, tarifas, bicicletas y su estado operativo. La solución adopta una arquitectura por capas en el backend â€"controlador, servicio, repositorio, mapper y DTOâ€" y un cliente Kotlin Multiplatform basado en Compose para web y Android. La documentación se elaboró a partir del código fuente vigente y se organiza conforme a pautas de presentación de APA, séptima edición.

Palabras clave: alquiler de bicicletas, Kotlin, Spring Boot, PostgreSQL, JWT, Compose Multiplatform.

2. DESCRIPCIÓN GENERAL

2.1. Propósito del sistema

Team Bici es una plataforma de alquiler de bicicletas diseñada para centralizar la operación de una flota distribuida en estaciones. Integra una API REST segura, una base de datos relacional y una interfaz multiplataforma. El sistema gestiona el ciclo de vida de usuarios, bicicletas, reservas, alquileres, tarifas y estaciones sin confundir una reserva con un alquiler efectivo.

2.2. Problema que resuelve

La gestión manual de alquileres puede ocasionar disponibilidad inconsistente, conflictos de horarios, pérdida de trazabilidad y cálculo impreciso de cobros. Team Bici resuelve estos riesgos mediante reglas de negocio transaccionales: verifica disponibilidad, identifica reservas superpuestas, evita que una bicicleta sea alquilada dos veces de manera simultánea, conserva el historial y calcula el costo final según el tiempo transcurrido.

2.3. Problema que resuelve

Registro e inicio de sesión con correo y contraseña.

- Autenticación mediante JWT y renovación de sesión con refresh tokens rotativos.
- Integración opcional de inicio de sesión con Google OAuth 2.0.
- Consulta de estaciones, tarifas y bicicletas disponibles.
- Creación, consulta y cancelación de reservas propias.
- Inicio de alquiler, devolución y cálculo prorrateado por minutos.
- Administración protegida de estaciones, tarifas, bicicletas y estados de la flota.

2.4. Tecnologías utilizadas

Capa	Tecnología	Finalidad
Backend	Kotlin 2.3.21 + Java 21	Lenguaje y plataforma de ejecución del servidor.
Framework	Spring Boot 4.1.1	Configuración, API REST, inyección de dependencias y ciclo de aplicación.
Persistencia	Spring Data JPA / Hibernate	Mapeo objeto-relacional y operaciones sobre entidades .
Base de datos	PostgreSQL	Persistencia relacional de usuarios, flota, reservas, alquileres y tokens.
Seguridad	Spring Security, BCrypt y JJWT 0.12.6	Autenticación, autorización por rol, hash de claves y tokens JWT.
OAuth	Spring Security OAuth2 Client	Inicio de sesión opcional con Google.
Frontend	Kotlin Multiplatform + Compose Multiplatform	Interfaz compartida para web y Android.
Red cliente	Ktor 3.3.3 + kotlinx.serialization	Consumo tipado de la API y serialización JSON.
Construcción	Gradle	Automatización de compilación, pruebas y ejecución.

3. ARQUITECTURA Y ESTRUCTURA DEL PROYECTO

3.1. Arquitectura backend por capas

El backend adopta una separación explícita de responsabilidades. El controlador expone contratos HTTP; el servicio aplica las reglas de negocio y las transacciones; el repositorio obtiene o guarda datos; el mapper convierte entre entidades y DTO; y las entidades representan el modelo

de persistencia. Esta división reduce acoplamiento, facilita pruebas y evita que las decisiones de negocio queden distribuidas en la capa web o de datos.

Directorio	Responsabilidad	Contenido relevante
api/dto	Contratos de entrada y salida de la API.	RequestDto, ResponseDto y objetos de autenticación.
controller	Recepción de solicitudes HTTP y delegación.	AuthController, EstacionController, BicicletaController, ReservaController y AlquilerController.
service	Interfaces de negocio.	AuthService, ReservaService, AlquilerService y servicios de catálogo.
service/impl	Implementaciones transaccionales.	Validación de disponibilidad, costos y sesiones.
repository	Acceso a PostgreSQL con Spring Data JPA.	Repositorios de entidades y consultas de solapamiento.
domain/entity	Modelo persistente JPA.	Usuario, Bicicleta, Reserva, Alquiler, Estacion, Tarifa y tokens.
domain/enums	Valores controlados de dominio.	Roles, estados y tipos de bicicleta.
mapper	Conversión entidad-DTO sin consultas.	Mappers de cada agregado.
security	Filtros, JWT, principal, handlers y configuración.	SecurityConfig, JwtService y JwtAuthenticationFilter.
config	Configuración transversal y tareas programadas.	CORS, administrador inicial y limpieza de tokens.
exception	Errores de dominio y respuestas normalizadas.	GlobalExceptionHandler y ErrorResponse.

3.2. Estructura fronted

El frontend se organiza como un proyecto Kotlin Multiplatform. shared concentra lógica y UI reutilizable; webApp contiene el punto de entrada del navegador; androidApp contiene el empaquetado Android. Las implementaciones específicas de almacenamiento y conversión de fechas se ubican en los source sets propios de cada plataforma.

3.3. Arquitectura backend por capas

El frontend se organiza como un proyecto Kotlin Multiplatform. shared concentra lógica y UI reutilizable; webApp contiene el punto de entrada del navegador; androidApp contiene el empaquetado Android. Las implementaciones específicas de almacenamiento y conversión de fechas se ubican en los source sets propios de cada plataforma.

4. COMPONENTES Y CLASES PRINCIPALES

4.1. Controladores REST

Clase	Ruta base	Responsabilidad
AuthController	/api/auth	Registro, inicio de sesión, renovación, cierre de sesión e intercambio OAuth2.
EstacionController	/api/estaciones	Consulta y gestión administrativa de estaciones.
TarifaController	/api/tarifas	Consulta y gestión administrativa de tarifas por tipo.
BicicletaController	/api/bicicletas	Consulta, alta, edición, disponibilidad y cambio de estado de bicicletas.
ReservaController	/api/reservas	Consulta, creación y cancelación de reservas del usuario autenticado.
AlquilerController	/api/alquileres	Consulta, inicio y devolución de alquileres propios.

4.2. Componentes de seguridad

Clase	Función
SecurityConfig	Define política stateless, rutas públicas, CORS, proveedor DAO, filtro JWT, manejo de 401/403 y OAuth opcional.
JwtService	Genera y valida JWT; incluye identidad, rol y versión de sesión.
JwtAuthenticationFilter	Extrae el Bearer token, carga el usuario y establece el contexto de seguridad si el token es válido.
CustomUserDetailsService	Convierte el usuario persistido en principal para Spring Security.
UsuarioPrincipal	Representa al usuario autenticado y sus autoridades en la capa de seguridad.
RestAuthenticationEntryPoint / RestAccessDeniedHandler	Devuelven respuestas JSON coherentes ante autenticación o autorización insuficiente.

4.3. Servicios de negocio

Componente	Función principal	Reglas relevantes
AuthServiceImpl	Gestiona identidad y emisión de credenciales.	Normaliza correo, verifica duplicados, cifra contraseñas y genera sesiones.
RefreshTokenService	Administra refresh tokens persistentes.	Almacena hashes, rota tokens, revoca familias y limpia tokens vencidos.
ReservaServiceImpl	Gestiona reservas temporales.	Exige periodo futuro válido, bicicleta disponible y ausencia de solapamientos confirmados.
AlquilerServiceImpl	Gestiona alquileres reales y devoluciones.	Exige tarifa activa, bloquea doble alquiler y calcula costo por minutos.
BicicletaServiceImpl	Gestiona catálogo y estado de flota.	Permite disponibilidad, mantenimiento y demás estados definidos.
EstacionServiceImpl	Gestiona estaciones.	Valida existencia y actualiza información administrativa.
TarifaServiceImpl	Gestiona precios.	Mantiene la tarifa por tipo de bicicleta.

4.4. Controladores REST

Componente	Responsabilidad
App	Compone repositorios, SessionManager, tema visual y navegación según sesión.
AuthScreen	Registro e inicio de sesión; permite mostrar u ocultar la contraseña.
SessionScreen	Cabecera de sesión y navegación por secciones según rol.
BicicletasScreen	Filtrado por estación, reserva y alquiler inmediato.
ReservationDateTimePicker	Calendario y selector visual de hora; convierte la hora local a UTC.
MisReservasScreen / MisAlquileresScreen	Consulta, cancelación y devolución del usuario.
AdministracionScreen	CRUD administrativo de estaciones, tarifas y bicicletas; cambio de estado y mantenimiento.
FeedbackBanner	Mensajes animados de confirmación y error.

5. MODELO DE DATOS Y PERSISTENCIA

PostgreSQL almacena el estado operativo. Las entidades JPA usan claves primarias Long generadas por identidad y las relaciones se modelan con claves foráneas. EntidadAuditable centraliza creado_en y actualizado_en para los registros que requieren trazabilidad temporal.

5.1. Entidades y campos principales

Tabla / entidad	Campos principales	Relaciones
usuarios / Usuario	id, nombre, apellido, correo, contrasena_hash, proveedor_auth, proveedor_id, google_subject, rol, activo, version_sesion.	Uno a muchos con reservas, alquileres y refresh tokens. Correo único.
estaciones / Estacion	id, codigo, nombre, direccion, latitud, longitud, activa.	Una estación contiene bicicletas y se vincula a retiros/devoluciones.
tarifas / Tarifa	id, tipo_bicicleta, valor_hora, activa.	Se consulta al iniciar un alquiler según el tipo de bicicleta.
bicicletas / Bicicleta	id, codigo, marca, modelo, tipo, estado, estacion_actual_id.	Pertenece a una estación; aparece en reservas y alquileres.
reservas / Reserva	id, usuario_id, bicicleta_id, estacion_retiro_id, fecha_inicio, fecha_fin, estado.	Muchos a uno con usuario, bicicleta y estación de retiro.
alquileres / Alquiler	id, usuario_id, bicicleta_id, estacion_retiro_id, estacion_devolucion_id, fechas, estado, tarifa_hora_aplicada, costo_final.	Muchos a uno con usuario, bicicleta y estaciones.
refresh_tokens / RefreshToken	id, token_hash, familia, usuario_id, expira_en, revocado_en.	Muchos a uno con usuario; controla renovación y revocación.
codigos_autorizacion_oauth / CodigoAutorizacionOAuth	id, codigo, usuario_id, expira_en, consumido_en.	Código temporal de un solo uso para intercambio OAuth.

5.2. Estados controlados

Enumeración	Valores	Uso
Rol	CLIENTE, ADMINISTRADOR	Autoriza capacidades de administración.
EstadoBicicleta	DISPONIBLE, RESERVADA, ALQUILADA, MANTENIMIENTO	Determina uso operativo de la flota.
EstadoReserva	PENDIENTE, CONFIRMADA, CANCELADA, COMPLETADA	Controla el ciclo de una reserva.
EstadoAlquiler	ACTIVO, FINALIZADO, CANCELADO	Controla el ciclo de un alquiler.
TipoBicicleta	URBANA, MONTANA, ELECTRICA, PLEGABLE	Clasifica bicicletas y relaciona tarifas.
ProveedorAuth	LOCAL, GOOGLE	Identifica el origen de la autenticación.

6. API REST y métodos importantes

La API se expone bajo /api. Salvo las rutas públicas de autenticación, todas requieren Authorization: Bearer <accessToken>. Las solicitudes y respuestas usan JSON. Los DTO de entrada se validan con Jakarta Bean Validation y los errores se devuelven mediante ErrorResponse.

6.1. Endpoints de autentificación

Método	Endpoint	Entrada	Función / salida
POST	/api/auth/registro	nombre, apellido, correo, contrasena	Crea cliente local y responde tokens + usuario. HTTP 201.
POST	/api/auth/login	correo, contrasena	Autentica y responde access token, refresh token y usuario.
POST	/api/auth/refresh	refreshToken	Rota el token de renovación y emite sesión actualizada.
POST	/api/auth/logout	Bearer token	Revoca sesiones activas del usuario. HTTP 204.
POST	/api/auth/oauth2/exchange	code	Intercambia código OAuth temporal por una sesión local.

6.2. Endpoints de catálogo y administración

Método	Endpoint	Función	Rol
GET	/api/estaciones	Lista estaciones.	Autenticado
GET	/api/estaciones/{id}	Obtiene una estación.	Autenticado
POST	/api/estaciones	Crea estación.	ADMINISTRADOR
PUT	/api/estaciones/{id}	Actualiza estación.	ADMINISTRADOR
DELETE	/api/estaciones/{id}	Elimina estación.	ADMINISTRADOR
GET	/api/tarifas	Lista tarifas.	Autenticado
POST / PUT	/api/tarifas [/{id}]	Crea o actualiza tarifa.	ADMINISTRADOR
GET	/api/bicicletas	Lista bicicletas.	Autenticado
GET	/api/bicicletas/disponibles?estacionId={id}	Filtra bicicletas disponibles.	Autenticado
POST / PUT	/api/bicicletas [/{id}]	Crea o actualiza bicicleta.	ADMINISTRADOR
PATCH	/api/bicicletas/{id}/estado?estado=MANTENIMIENTO	Actualiza estado de flota.	ADMINISTRADOR

6.3. Endpoints de reservas y alquileres

Método	Endpoint	Entrada	Función
GET	/api/reservas	Token de autentificación	Lista reservas del usuario autenticado.
POST	/api/reservas	bicicletaId, estacionRetiroId, fechaInicio, fechaFin	Crea reserva si el periodo futuro estÃ¡ libre.
PATCH	/api/reservas/{id}/cancelar	Token de autentificación	Cancela una reserva propia aún futura.
GET	/api/alquileres	Token de autentificación	Lista alquileres del usuario autenticado.
POST	/api/alquileres	bicicletaId, estacionRetiroId	Inicia alquiler para bicicleta disponible con tarifa activa.
PATCH	/api/alquileres/{id}/devolucion	estacionDevolucionId	Finaliza alquiler, calcula costo y actualiza ubicación.

6.4. Ejemplos de métodos críticos

AuthController.registro recibe un RegistroRequestDto validado. Delegar al servicio impide que el controlador contenga cifrado, comprobación de duplicados o generación de tokens. La salida es un AuthResponseDto con credenciales y datos seguros del usuario; nunca se devuelvecontrasenaHash.

```
@PostMapping("/registro") @ResponseStatus(HttpStatus.CREATED)
fun registro(@Valid @RequestBody dto: RegistroRequestDto) = service.registrar(dto)
```

ReservaServiceImpl.crear recibe el identificador del usuario autenticado y el DTO. Verifica que fechaFin sea posterior a fechaInicio, que el inicio sea futuro, que la bicicleta estÃ© disponible en la estaciÃ³n indicada y que no exista solapamiento confirmado. Devuelve ReservaResponseDto.

AlquilerServiceImpl.devolver recibe identificador de alquiler, propietario autenticado y estación de devolución. Comprueba propiedad y estado ACTIVO; calcula los minutos entre fechas, multiplica por tarifaHoraAplicada, divide entre 60 y redondea a dos decimales con HALF_UP. Luego ubica la bicicleta en la estación destino y la deja DISPONIBLE

7. HISTORIAS DE USUARIO

7.1. Autenticación local y BCrypt

Las contraseñas no se persisten en texto plano. AuthServiceImpl usa PasswordEncoder, configurado como BCryptPasswordEncoder, antes de crear un Usuario. BCrypt incorpora sal y un factor de trabajo, de modo que una misma contraseña genera hashes diferentes y el costo de ataques por fuerza bruta se incrementa (Spring Security, s. f.).

7.2. JWT y control de sesión

Tras un registro, login u operación OAuth válida, JwtService emite un access token firmado. El token identifica el correo, el usuario, el rol y la versión de sesión. JwtAuthenticationFilter extrae el encabezado Bearer, valida la firma y verifica la versión de sesión contra el usuario actual. Si el usuario cierra sesión, la versión se incrementa, invalidando lógicamente los access tokens anteriores.

7.3. Refresh tokens rotativos

El refresh token no se guarda en texto claro; se guarda su hash SHA-256. Cada token pertenece a una familia. Al renovar, el token usado queda revocado y se emite uno nuevo. Una reutilización o token inválido revoca la familia como mecanismo de detección de replay. El trabajo TokenCleanupJob elimina tokens expirados o revocados antiguos de acuerdo con TOKEN_CLEANUP_CRON.

7.4. Autorización y CORS

SecurityConfig deshabilita CSRF porque la API usa autenticación Bearer y política stateless. Todas las rutas requieren autenticación salvo registro, login, refresh y los recursos OAuth necesarios. Las anotaciones @PreAuthorize restringen las operaciones de administración al rol ADMINISTRADOR. CorsConfig permite únicamente los orígenes definidos por FRONTEND_ALLOWED_ORIGINS; en desarrollo incluye usualmente http://localhost:8081.

7.5. Errores seguros

GlobalExceptionHandler transforma errores de validación, recurso inexistente, reglas de negocio, datos inválidos y errores inesperados a ErrorResponse con status, message, timestamp, errors opcional y path. RestAuthenticationEntryPoint y RestAccessDeniedHandler devuelven JSON para 401 y 403, evitando respuestas HTML inconsistentes.

8. APLICACIÓN CLIENTE Y EXPERIENCIA DE USUARIO

8.1. Comunicación y sesión

ApiHttpClient centraliza el cliente Ktor, serialización JSON, encabezados Bearer y el proceso de refresh. AuthRepository encapsula registro, login, intercambio OAuth y logout. SessionManager mantiene el estado observable de AuthSession. Para web, SessionStorage usa sessionStorage del navegador; la sesión persiste durante la pestaña y se borra al cerrarla. En Android, el almacenamiento actual es de memoria y debe sustituirse por una solución cifrada antes de producción.

8.2. Experiencia de reserva

ReservationDateTimePicker reemplaza la entrada manual de fechas ISO. El componente integra DatePicker y TimePicker de Material 3, conserva la fecha y hora seleccionadas localmente y las convierte a UTC mediante implementaciones específicas de plataforma antes de enviar la reserva. La validación exige horas completas y que la fecha final sea posterior a la inicial.

8.3. Diseño y retroalimentación

El tema Material 3 usa una paleta índigo/violeta como identidad principal. FeedbackBanner presenta confirmaciones y errores con transiciones de aparición, expansión, salida y contracción. AdministracionScreen usa contenido animado al cambiar entre estaciones, tarifas y bicicletas; las tarjetas ajustan su contenido con animación. Los formularios de estación validan latitud entre -90 y 90 y longitud entre -180 y 180 antes de llamar a la API.

8.4. Accesibilidad y mejoras previstas

Mantener etiquetas explícitas en campos, botones y selectores.

- No depender exclusivamente del color: los estados incluyen texto descriptivo.
- Implementar almacenamiento cifrado con Android Keystore y DataStore antes de publicación móvil.
- Agregar pruebas de interfaz, selector de estaciones escalable y paginación cuando la flota crezca.
- Incorporar recursos gráficos propios o con licencia documentada, conservando texto alternativo y atribución cuando aplique.

9. APLICACIÓN CLIENTE Y EXPERIENCIA DE USUARIO

9.1. Requisitos

Componente	Requisito
Backend	JDK 21, Gradle Wrapper incluido y PostgreSQL disponible.
Base de datos	PostgreSQL local con base de datos y credenciales definidas.
Frontend	JDK compatible con Gradle; navegador moderno para Compose web.
Herramientas recomendadas	IntelliJ IDEA con soporte Kotlin/Spring y cliente PostgreSQL.

9.2. Variables de entorno del backend

El archivo application.yaml importa opcionalmente .env. El archivo .env no debe versionarse; .env.example sirve como plantilla. Las variables de seguridad deben configurarse con valores reales y secretos fuera del control de versiones

Variable	Descripción	Ejemplo de desarrollo
DB_HOST, DB_PORT, DB_NAME	Ubicación y nombre de PostgreSQL.	localhost, 5432, team_bici
DB_USERNAME, DB_PASSWORD	Credenciales de base de datos.	postgres, valor local
JPA_DDL_AUTO	Estrategia de esquema JPA.	update (desarrollo); validate (producción)
SERVER_PORT	Puerto HTTP del backend.	8080
JWT_SECRET	Secreto de firma de al menos 32 caracteres.	Valor aleatorio y confidencial
JWT_EXPIRATION_MINUTES	Duración del access token.	60
JWT_REFRESH_EXPIRATION_DAYS	Duración del refresh token.	14
ADMIN_EMAIL, ADMIN_PASSWORD	Datos para administrador inicial.	Solo uso inicial y seguro

SPRING_PROFILES_ACTIVE	Perfil para funcionalidades opcionales.	oauth2
GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET	Credenciales OAuth de Google.	Valores de Google Cloud
FRONTEND_ALLOWED_ORIGINS	Orígenes permitidos por CORS.	http://localhost:8081

9.3. Ejecución del backend

```
cd C:\AlquilerBicicletasBk\team-bici-backend
Copy-Item .env.example .env
# Editar .env con credenciales reales y JWT_SECRET seguro
.\gradlew.bat test --no-daemon
.\gradlew.bat bootRun
```

El servicio se inicia en http://localhost:8080, salvo que SERVER_PORT indique otro puerto. Para una base existente, las modificaciones de esquema deben gestionarse con migraciones versionadas; depender de ddl-auto=update no sustituye una estrategia de migración de producción.

9.4. Ejecución del frontend

```
cd C:\AlquilerBicicletasBk\team-bici-frontend
.\gradlew.bat :webApp:jsBrowserDevelopmentRun
# Android (ensamblado de depuración)
.\gradlew.bat :androidApp:assembleDebug
```

El servicio se inicia en http://localhost:8080, salvo que SERVER_PORT indique otro puerto. Para una base existente, las modificaciones de esquema deben gestionarse con migraciones versionadas; depender de ddl-auto=update no sustituye una estrategia de migración de producción.

10. CONVENCIONES DE CÓDIGO Y COMENTARIOS

10.1. Convenciones aplicadas

Los paquetes Kotlin están en minúsculas bajo com.alquilerbicicletas.teambicibackend

- Las clases usan PascalCase; funciones y propiedades, camelCase.
- Las enumeraciones representan estados limitados y evitan cadenas inconsistentes.
- Los DTO separan la entrada de la salida para limitar quÃ© campos puede enviar el cliente.
- Los servicios son el lugar de reglas de negocio y transacciones; los controladores se mantienen delgados.
- Los mappers no consultan repositorios; reciben las entidades ya resueltas por el servicio.

10.2. Política de comentarios

No se debe comentar cada línea ni repetir lo que el código expresa con claridad. Los comentarios se reservan para decisiones no obvias, restricciones de seguridad, reglas de negocio de difícil deducción, razones de compatibilidad y efectos secundarios. Por ejemplo, la rotación de refresh tokens y la conversión de hora local a UTC son candidatos a comentarios breves porque responden a riesgos de seguridad o integridad temporal.

10.3. Ejemplo de comentario útil

// Reutilizar un refresh token indica posible robo; se revoca toda la familia.

revokeFamily(token.familia)

Un comentario como se asigna el usuario□ antes de usuario = ... no agrega información y debe evitarse. La claridad principal debe provenir de nombres precisos, funciones pequeñas y tipos apropiados.

11. VALIDACIÓN, ERRORES Y SONSIDERACIONES OPERATIVAS

11.1. Validación de datos

Los DTO del backend aplican restricciones de Jakarta Validation. Por ejemplo, EstacionRequestDto exige código, nombre y dirección no vacíos; latitud entre -90 y 90; y longitud entre -180 y 180. El cliente replica estas reglas críticas para entregar retroalimentación inmediata, pero el backend permanece como fuente de verdad.

11.2. Reglas de integridad destacadas

No se permite reservar en el pasado ni con fecha final igual o anterior al inicio.

- Una bicicleta debe estar DISPONIBLE y en la estación indicada para reservarse o alquilarse.
- No se permite superponer reservas CONFIRMADA para la misma bicicleta.
- Para alquilar debe existir una tarifa activa del tipo de bicicleta.
- Solo el propietario puede cancelar su reserva o devolver su alquiler.
- Una estación de devolución debe estar activa.
- Las operaciones de catálogo administrativo requieren rol ADMINISTRADOR

11.3. Operación y evolución recomendada

Adoptar Flyway o Liquibase para migraciones de PostgreSQL versionadas

- . Separar secretos por entorno y usar un gestor de secretos en producción.
- Configurar HTTPS, encabezados de seguridad y monitoreo en despliegues públicos.

- Añadir pruebas de integración para flujos de concurrencia de reservas y alquileres.
- Incorporar registro estructurado y métricas para diagnosticar errores sin exponer secretos.
- Revisar periódicamente dependencias y configurar copias de seguridad de PostgreSQL.

12. REFERENCIAS

Apache Software Foundation. (s. f.). Gradle user manual. https://docs.gradle.org/

JetBrains. (s. f.). Kotlin documentation. https://kotlinlang.org/docs/home.html

JetBrains. (s. f.). Compose Multiplatform documentation. https://www.jetbrains.com/help/kotlin-multiplatform-dev/compose-multiplatform.html

Ktor. (s. f.). Ktor documentation. https://ktor.io/docs/

OWASP Foundation. (s. f.). JSON Web Token for Java cheat sheet. https://cheatsheetseries.owasp.org/

PostgreSQL Global Development Group. (s. f.). PostgreSQL documentation. https://www.postgresql.org/docs/

Spring. (s. f.). Spring Boot reference documentation. https://docs.spring.io/spring-boot/

Spring. (s. f.). Spring Security reference. https://docs.spring.io/spring-security/